AISSMS

INSTITUTE OF INFORMATION TECHNOLOGY

ADDING VALUE TO ENGINEERING

Department of Information Technology

Lab Manual

Deep Learning (ITPEC609)

Third Year

Information Technology

Vision

To be a leader in preparing technically competent and skillful IT Graduates to address the needs of industry and society.

Mission

curricular, extracurricular and extension activities.

ii. To promote research and professional activities through industry involvement and professional bodies

iii. To instil professional ethics and lifelong learning skills with concern for the society.

Programme Educational Objectives

Graduates will

i. Excel in diverse career paths with core professional skills.

ii. Engage in multi domain research/professional activities.

iii. Cater to the needs of society with IT solutions/applications.

Program Specific Outcomes

Graduates will be able to

1. Use database, networking and programming technologies for solving real life problems.
2. Develop applications in the field of computing, networking, security and analytics.

Program Outcomes

Graduates will be able to

1. Apply the knowledge of mathematics, science, engineering fundamentals, and an engineering specialization to the solution of complex engineering problems.**[Engineering knowledge]**
2. Identify, formulate, research literature, and analyse complex engineering problems reaching substantiated conclusions using first principles of mathematics, natural sciences, and engineering sciences.**[Problem analysis]**
3. Design solutions for complex engineering problems and design system components or processes that meet the specified needs with appropriate consideration for the public health and safety, and the cultural, societal, and environmental considerations.**[Design/development of solutions]**

4. Use research-based knowledge and research methods including design of experiments, analysis and interpretation of data, and synthesis of the information to provide valid conclusions.**[Conduct investigations of complex problems]**
5. Create, select, and apply appropriate techniques, resources, and modern engineering and IT tools including prediction and modelling to complex engineering activities with an understanding of the limitations. **[Modern tool usage]**
6. Apply reasoning informed by the contextual knowledge to assess societal, health, safety, legal and cultural issues and the consequent responsibilities relevant to the professional engineering practice.**[The engineer and society]**
7. Understand the impact of the professional engineering solutions in societal and environmental contexts, and demonstrate the knowledge of, and need for sustainable development.**[Environment and sustainability]**
8. Apply ethical principles and commit to professional ethics and responsibilities and norms of the engineering practice.**[Ethics]**
9. Function effectively as an individual, and as a member or leader in diverse teams, and in multidisciplinary settings.**[Individual and team work]**
10. Communicate effectively on complex engineering activities with the engineering community and with society at large, such as, being able to comprehend and write effective reports and design documentation, make effective presentations, and give and receive clear instructions.**[Communication]**
11. Demonstrate knowledge and understanding of the engineering and management principles and apply these to one's own work, as a member and leader in a team, to manage projects and in multidisciplinary environments.[Project management and finance]
12. Recognize the need for, and have the preparation and ability to engage in independent and life-long learning in the broadest context of technological change.[Life-long learning]

Third Year Information Technology (2025 Course) Software Lab II - Deep Learning			
Course Code:	**ITPEC609**	**Credit:**	**2**
Contact Hours:	4 Hrs./week(P)	**Type of Course:**	Practical
Examination Scheme	Term-work 25 Marks	Practical 25 Marks	

Pre-requisites:

1. Machine Learning

Course assessment methods/tools:

Sr. No.	Course assessment methods/tools	External/ Internal	Marks
1.	Term work	Internal	25
2.	Practical	External	25

Course Objectives	
1	To learn the fundamentals and application of Deep Learning methodologies
2	To identify deep learning algorithm to solve real world problems
3	To design and develop an application-specific deep learning model

Course Outcomes: Students will be able to	
609.1	Implement neural networks with Python
609.2	Develop Deep Leaning applications based using CNN, RNN, LSTM
609.3	Develop deep convolutional generative adversarial network
609.4	Evaluate the performance of the model build using Deep Learning

List of Experiments

Sr. No.	Problem Statement	Page No.
1	Implement simple vector addition in TensorFlow	6
2	Implement multi-layer perceptron neural network with Python and NumPy. Use the modern deep learning framework Py-Torch to analyze real data	8
3	Develop convolution neural networks and use them to classify images (faces, melanomas, etc.) based on patterns and objects that appear in them	12
4	Develop recurrent networks (RNN) and long short-term memory (LSTM) networks with Py-Torch; perform sentiment analysis	15
5	Implement the Deep Convolutional Generative Adversarial Network (DCGAN) model to simulate realistic images, with In Goodfellow, the inventor of GANs	18
6	Implement deep neural networks to design agents to take actions in a simulated environment.	22

Experiment No. 1

Aim:
To implement simple vector addition using TensorFlow.

Problem Statement:
Implement simple vector addition in TensorFlow

Objectives:
- To understand the basics of TensorFlow operations.
- To learn how tensors are created.
- To perform element-wise vector addition.
- To execute and verify results using TensorFlow.

Concepts:
- Tensor
- Vector
- Element-wise addition
- Computational graph
- Eager execution

Theory:
TensorFlow is an open-source machine learning framework developed by Google. It is widely used for numerical computation and building deep learning models.

A **tensor** is a multi-dimensional array. A vector is a 1-dimensional tensor.

If two vectors are:

$$A = [a_1, a_2, a_3] B = [b_1, b_2, b_3]$$

Then their vector addition is:

$$C = A + B = [a_1 + b_1, a_2 + b_2, a_3 + b_3]$$

TensorFlow performs this operation efficiently using built-in tensor operations.

Procedure:
1. Install TensorFlow using pip:

   pip install tensorflow
2. Import TensorFlow library:

   import tensorflow as tf
3. Create two vectors (1D tensors):

```
vector1 = tf.constant([1, 2, 3])
vector2 = tf.constant([4, 5, 6])
```

4. Perform vector addition:

```
result = tf.add(vector1, vector2)
```

5. Display the result:

```
print("Vector Addition Result:", result.numpy())
```

Observation:

Input:
Vector 1 = [1, 2, 3]

Vector 2 = [4, 5, 6]

Output:
Vector Addition Result = [5, 7, 9]

The output confirms that TensorFlow correctly performs element-wise vector addition.

Conclusion:

The experiment successfully demonstrated simple vector addition using TensorFlow. It helped in understanding tensors, basic operations, and execution of numerical computations using TensorFlow. This forms the foundation for more complex operations used in machine learning and deep learning models.

Experiment No. 2

Aim:

To implement a Multi-Layer Perceptron (MLP) neural network using Python and NumPy, and to analyse real-world data using Py-Torch.

Problem Statement:

Implement multi-layer perceptron neural network with Python and NumPy. Use the modern deep learning framework Py-Torch to analyse real data

Objectives:

- To understand the architecture of a Multi-Layer Perceptron (MLP).
- To implement forward propagation using NumPy.
- To understand activation functions and backpropagation.
- To train and evaluate a neural network using PyTorch on real-world data.

Concepts:

- Artificial Neural Network (ANN)
- Multi-Layer Perceptron (MLP)
- Forward Propagation
- Backpropagation
- Activation Functions (ReLU, Sigmoid, Softmax)
- Loss Function (Cross-Entropy Loss)
- Optimizer (SGD/Adam)

Theory:

A Multi-Layer Perceptron (MLP) is a class of feedforward artificial neural network consisting of:

- Input Layer
- One or more Hidden Layers
- Output Layer

Each neuron performs:

$$Z = WX + bA = f(Z)$$

Where:

- **W** = weights
- **X** = input

- **b** = bias
- **f** = activation function

Backpropagation is used to minimize the loss by updating weights using gradient descent.

PyTorch, developed by Meta Platforms, is widely used for deep learning because of its dynamic computation graph and flexibility.

Procedure:

Part A: Implement MLP using Python and NumPy

1. Import libraries:

```
import numpy as np
```

2. Define activation function:

```
def relu(x):
    return np.maximum(0, x)
```

3. Initialize inputs and weights:

```
X = np.array([[0,0], [0,1], [1,0], [1,1]])
y = np.array([[0], [1], [1], [0]])

np.random.seed(42)
W1 = np.random.randn(2, 4)
b1 = np.zeros((1,4))
W2 = np.random.randn(4, 1)
b2 = np.zeros((1,1))
```

4. Forward propagation:

```
Z1 = np.dot(X, W1) + b1
A1 = relu(Z1)

Z2 = np.dot(A1, W2) + b2
output = 1 / (1 + np.exp(-Z2))  # Sigmoid
```

5. Print output:

```
print("Output:\n", output)
```

Part B: Implement MLP using PyTorch on Real Data

1. Import libraries:

```
import torch
import torch.nn as nn
import torch.optim as optim
from torchvision import datasets, transforms
```

2. Load dataset (MNIST):

```
transform = transforms.ToTensor()
train_data = datasets.MNIST(root='./data', train=True, download=True,
transform=transform)

train_loader = torch.utils.data.DataLoader(train_data, batch_size=64, shuffle=True)
```

3. Define MLP Model:

```
class MLP(nn.Module):
    def __init__(self):
        super(MLP, self).__init__()
        self.fc1 = nn.Linear(784, 128)
        self.relu = nn.ReLU()
        self.fc2 = nn.Linear(128, 10)

    def forward(self, x):
        x = x.view(-1, 784)
        x = self.relu(self.fc1(x))
        x = self.fc2(x)
        return x

model = MLP()
```

4. Define Loss and Optimizer:

```
criterion = nn.CrossEntropyLoss()
optimizer = optim.Adam(model.parameters(), lr=0.001)
```

5. Training Loop:

```
for epoch in range(3):
    for images, labels in train_loader:
        outputs = model(images)
        loss = criterion(outputs, labels)

        optimizer.zero_grad()
        loss.backward()
        optimizer.step()

    print(f"Epoch {epoch+1}, Loss: {loss.item()}")
```

Observation:

- NumPy implementation successfully performs forward propagation.
- PyTorch model trains on MNIST dataset.
- Loss decreases over epochs.
- Model achieves good classification accuracy (~95% after sufficient epochs).

Conclusion:

The experiment successfully implemented a Multi-Layer Perceptron using Python and NumPy, demonstrating the basic neural network structure and forward propagation. Using PyTorch, the neural network was trained on real-world MNIST data, showing effective learning and classification performance. This experiment provides practical understanding of neural network implementation from scratch and using modern deep learning frameworks.

Experiment No. 3

Aim:

To develop a Convolutional Neural Network (CNN) and use it to classify images (such as faces and melanomas) based on visual patterns and objects.

Problem Statement:

Develop convolution neural networks and use them to classify images (faces, melanomas, etc.) based on patterns and objects that appear in them

Objectives:

- To understand the architecture of Convolutional Neural Networks (CNNs).
- To learn about convolution, pooling, and fully connected layers.
- To implement a CNN model using PyTorch.
- To train the CNN for image classification tasks.
- To evaluate model performance using accuracy and loss metrics.

Concepts:

- Convolutional Neural Network (CNN)
- Convolution Layer
- Kernel / Filter
- Feature Map
- Pooling Layer (Max Pooling)
- Fully Connected Layer
- Activation Function (ReLU, Softmax)
- Loss Function (Cross Entropy)
- Backpropagation

Theory:

A Convolutional Neural Network (CNN) is a type of deep learning model specially designed for image processing tasks. Unlike traditional neural networks, CNNs automatically detect important features such as edges, textures, and shapes from images.

Main Components of CNN:

1. **Convolution Layer:**

   Applies filters (kernels) to extract features from input images.

2. **Activation Function (ReLU):**

   Introduces non-linearity:

$$f(x) = max(0, x)$$

3. **Pooling Layer:**

   Reduces spatial dimensions (commonly Max Pooling).

4. **Fully Connected Layer:**

   Performs classification based on extracted features.

CNNs are widely used in medical image classification (e.g., melanoma detection) and face recognition systems.

Procedure:

1. **Import Required Libraries**

```
import torch
import torch.nn as nn
import torch.optim as optim
from torchvision import datasets, transforms
```

2. **Data Preprocessing**

```
transform = transforms.Compose([
    transforms.Resize((64, 64)),
    transforms.ToTensor()
])

train_data = datasets.ImageFolder(root='./dataset/train', transform=transform)
train_loader = torch.utils.data.DataLoader(train_data, batch_size=32, shuffle=True)
```

3. **Define CNN Model**

```
class CNN(nn.Module):
    def __init__(self):
        super(CNN, self).__init__()

        self.conv1 = nn.Conv2d(3, 32, kernel_size=3)
        self.pool = nn.MaxPool2d(2, 2)
        self.conv2 = nn.Conv2d(32, 64, kernel_size=3)

        self.fc1 = nn.Linear(64 * 14 * 14, 128)
        self.fc2 = nn.Linear(128, 2)  # Example: 2 classes

        self.relu = nn.ReLU()

    def forward(self, x):
        x = self.pool(self.relu(self.conv1(x)))
        x = self.pool(self.relu(self.conv2(x)))
        x = x.view(x.size(0), -1)
        x = self.relu(self.fc1(x))
```

```
        x = self.fc2(x)
        return x

model = CNN()
```

4. **Define Loss and Optimizer**

```
criterion = nn.CrossEntropyLoss()
optimizer = optim.Adam(model.parameters(), lr=0.001)
```

5. **Train the Model**

```
for epoch in range(5):
    running_loss = 0.0

    for images, labels in train_loader:
        outputs = model(images)
        loss = criterion(outputs, labels)

        optimizer.zero_grad()
        loss.backward()
        optimizer.step()

        running_loss += loss.item()

    print(f"Epoch {epoch+1}, Loss: {running_loss/len(train_loader)}")
```

Observation:

- The CNN successfully extracts spatial features from images.
- Loss decreases with each epoch during training.
- The model learns to classify images based on visual patterns.
- Accuracy improves with more training data and epochs.

For medical datasets (e.g., melanoma detection), proper preprocessing and sufficient training data significantly improve model performance.

Conclusion:

The experiment successfully demonstrated the development of a Convolutional Neural Network for image classification. CNNs automatically extract important features such as edges and textures, making them highly effective for tasks like face recognition and melanoma detection.

This experiment provides practical understanding of deep learning in computer vision and shighlights the importance of CNNs in real-world image analysis applications.

Experiment No. 4

Aim:

To develop Recurrent Neural Networks (RNN) and Long Short-Term Memory (LSTM) networks using PyTorch and perform sentiment analysis on text data.

Problem Statement:

Develop recurrent networks (RNN) and long short-term memory (LSTM) networks with Py-Torch; perform sentiment analysis

Objectives:

- To understand sequence modeling using RNN and LSTM.
- To implement RNN and LSTM architectures using PyTorch.
- To preprocess textual data for sentiment analysis.
- To train and evaluate models on real-world sentiment datasets.

Concepts:

- Recurrent Neural Network (RNN)
- Long Short-Term Memory (LSTM)
- Hidden State
- Vanishing Gradient Problem
- Word Embedding
- Tokenization
- Binary Classification
- Cross-Entropy Loss

Theory:

Recurrent Neural Networks (RNNs) are designed for sequential data such as text, speech, and time-series data. Unlike feedforward networks, RNNs maintain a **hidden state** that captures information from previous inputs.

However, standard RNNs suffer from the **vanishing gradient problem**, making it difficult to learn long-term dependencies.

LSTM (Long Short-Term Memory) networks overcome this limitation using special gates:

1. **Forget Gate** – Decides what information to discard.
2. **Input Gate** – Decides what new information to store.
3. **Output Gate** – Determines the output from the cell state.

In sentiment analysis, the model predicts whether a sentence expresses positive or negative sentiment.

Procedure:

1. Import Libraries
```
import torch
import torch.nn as nn
import torch.optim as optim
from torchtext.datasets import IMDB
from torchtext.data.utils import get_tokenizer
from torch.nn.utils.rnn import pad_sequence
```
2. Data Preprocessing
```
tokenizer = get_tokenizer("basic_english")

def yield_tokens(data_iter):
    for label, text in data_iter:
        yield tokenizer(text)

train_iter = IMDB(split='train')
```
Convert text into numerical form (vocabulary indexing) and pad sequences.
3. Define RNN Model
```
class RNNModel(nn.Module):
    def __init__(self, vocab_size, embed_dim, hidden_dim, output_dim):
        super(RNNModel, self).__init__()
        self.embedding = nn.Embedding(vocab_size, embed_dim)
        self.rnn = nn.RNN(embed_dim, hidden_dim, batch_first=True)
        self.fc = nn.Linear(hidden_dim, output_dim)

    def forward(self, x):
        embedded = self.embedding(x)
        output, hidden = self.rnn(embedded)
        return self.fc(hidden.squeeze(0))
```
4. Define LSTM Model
```
class LSTMModel(nn.Module):
    def __init__(self, vocab_size, embed_dim, hidden_dim, output_dim):
        super(LSTMModel, self).__init__()
        self.embedding = nn.Embedding(vocab_size, embed_dim)
        self.lstm = nn.LSTM(embed_dim, hidden_dim, batch_first=True)
        self.fc = nn.Linear(hidden_dim, output_dim)

    def forward(self, x):
        embedded = self.embedding(x)
        output, (hidden, cell) = self.lstm(embedded)
        return self.fc(hidden.squeeze(0))
```
5. Training Setup

```
model = LSTMModel(vocab_size=10000, embed_dim=100, hidden_dim=128,
output_dim=2)

criterion = nn.CrossEntropyLoss()
optimizer = optim.Adam(model.parameters(), lr=0.001)
```

6. Training Loop

```
for epoch in range(5):
    total_loss = 0

    for text, labels in train_loader:
        outputs = model(text)
        loss = criterion(outputs, labels)

        optimizer.zero_grad()
        loss.backward()
        optimizer.step()

        total_loss += loss.item()

    print(f"Epoch {epoch+1}, Loss: {total_loss}")
```

Observation:

- RNN model learns basic sentiment patterns but struggles with long sentences.
- LSTM performs better due to memory gates handling long-term dependencies.
- Training loss decreases over epochs.
- LSTM achieves higher accuracy compared to simple RNN in sentiment classification tasks.

Conclusion:

The experiment successfully implemented RNN and LSTM models using PyTorch for sentiment analysis. RNN captures sequential information but suffers from vanishing gradients. LSTM overcomes this issue using gating mechanisms, leading to improved performance.

This experiment demonstrates the importance of sequence models in Natural Language Processing (NLP) tasks such as sentiment analysis.

Experiment No. 5

Aim:

To implement a Deep Convolutional Generative Adversarial Network (DCGAN) model to generate realistic images using PyTorch.

Problem Statement:

Implement the Deep Convolutional Generative Adversarial Network (DCGAN) model to simulate realistic images, with In Goodfellow, the inventor of GANs

Objectives:

- To understand the working principle of Generative Adversarial Networks (GANs).
- To implement DCGAN architecture using convolutional layers.
- To train Generator and Discriminator networks.
- To generate realistic synthetic images from random noise.
- To analyze training stability and generated outputs.

Concepts:

- Generative Adversarial Network (GAN)
- Generator
- Discriminator
- Latent Space
- Convolutional Layers
- Transposed Convolution
- Binary Cross-Entropy Loss
- Adversarial Training

Theory:

A Generative Adversarial Network (GAN) consists of two neural networks:

1. **Generator (G):**

   Generates fake images from random noise (latent vector).

2. **Discriminator (D):**

   Classifies images as real or fake.

Both networks compete in a **minimax game**:

$$min_G max_D V(D, G)$$

The Generator tries to fool the Discriminator, while the Discriminator tries to correctly distinguish real from fake images.

DCGAN improves traditional GANs by:

- Using convolutional layers instead of fully connected layers
- Using batch normalization
- Removing fully connected hidden layers
- Using ReLU in Generator and LeakyReLU in Discriminator

Procedure:

1. Import Required Libraries

```
import torch
import torch.nn as nn
import torch.optim as optim
import torchvision.datasets as datasets
import torchvision.transforms as transforms
import torchvision.utils as vutils
```

2. Load Dataset (Example: MNIST)

```
transform = transforms.Compose([
    transforms.Resize(64),
    transforms.ToTensor(),
    transforms.Normalize((0.5,), (0.5,))
])

dataset = datasets.MNIST(root='./data', download=True, transform=transform)
dataloader = torch.utils.data.DataLoader(dataset, batch_size=128, shuffle=True)
```

3. Define Generator

```
class Generator(nn.Module):
    def __init__(self):
        super(Generator, self).__init__()
        self.main = nn.Sequential(
            nn.ConvTranspose2d(100, 512, 4, 1, 0, bias=False),
            nn.BatchNorm2d(512),
            nn.ReLU(True),

            nn.ConvTranspose2d(512, 256, 4, 2, 1, bias=False),
            nn.BatchNorm2d(256),
            nn.ReLU(True),

            nn.ConvTranspose2d(256, 1, 4, 2, 1, bias=False),
            nn.Tanh()
        )

    def forward(self, x):
        return self.main(x)
```

4. Define Discriminator

```
class Discriminator(nn.Module):
    def __init__(self):
        super(Discriminator, self).__init__()
        self.main = nn.Sequential(
            nn.Conv2d(1, 256, 4, 2, 1, bias=False),
            nn.LeakyReLU(0.2, inplace=True),

            nn.Conv2d(256, 512, 4, 2, 1, bias=False),
            nn.BatchNorm2d(512),
            nn.LeakyReLU(0.2, inplace=True),

            nn.Conv2d(512, 1, 4, 1, 0, bias=False),
            nn.Sigmoid()
        )

    def forward(self, x):
        return self.main(x).view(-1)
```

5. Loss Function and Optimizers

```
netG = Generator()
netD = Discriminator()

criterion = nn.BCELoss()

optimizerD = optim.Adam(netD.parameters(), lr=0.0002, betas=(0.5, 0.999))
optimizerG = optim.Adam(netG.parameters(), lr=0.0002, betas=(0.5, 0.999))
```

6. Training Loop

```
for epoch in range(5):
    for i, (real_images, _) in enumerate(dataloader):

        # Train Discriminator
        netD.zero_grad()
        real_labels = torch.ones(real_images.size(0))
        fake_labels = torch.zeros(real_images.size(0))

        output_real = netD(real_images)
        loss_real = criterion(output_real, real_labels)

        noise = torch.randn(real_images.size(0), 100, 1, 1)
        fake_images = netG(noise)

        output_fake = netD(fake_images.detach())
        loss_fake = criterion(output_fake, fake_labels)
```

```
lossD = loss_real + loss_fake
lossD.backward()
optimizerD.step()

# Train Generator
netG.zero_grad()
output = netD(fake_images)
lossG = criterion(output, real_labels)
lossG.backward()
optimizerG.step()

print(f"Epoch {epoch+1}, Loss D: {lossD.item()}, Loss G: {lossG.item()}")
```

Observation:

- The Generator gradually improves in producing realistic-looking digits.
- The Discriminator initially performs well but becomes less confident as Generator improves.
- Loss values fluctuate due to adversarial training dynamics.
- Generated images become sharper with more epochs.

Conclusion:

The experiment successfully implemented a Deep Convolutional GAN using PyTorch. The adversarial training between Generator and Discriminator enabled the model to generate realistic synthetic images.

This experiment demonstrates the power of GANs in image synthesis and highlights their applications in image generation, data augmentation, and creative AI systems.

Experiment No. 6

Aim:

To implement Deep Neural Networks to design intelligent agents that take actions in a simulated environment.

Problem Statement:

Implement deep neural networks to design agents to take actions in a simulated environment

Objectives:

- To understand the concept of Reinforcement Learning (RL).
- To design an agent that interacts with an environment.
- To implement Deep Q-Network (DQN) using PyTorch.
- To train the agent to maximize cumulative reward.
- To analyze the agent's learning performance.

Concepts:

- Reinforcement Learning (RL)
- Agent and Environment
- State, Action, Reward
- Policy
- Q-Learning
- Deep Q-Network (DQN)
- Exploration vs Exploitation
- Experience Replay

Theory:

Reinforcement Learning is a learning paradigm where an **agent** interacts with an **environment** and learns to take actions to maximize cumulative reward.

At each time step:

- Agent observes **state (S)**
- Takes **action (A)**
- Receives **reward (R)**
- Moves to next **state (S')**

The goal is to learn an optimal policy:

$$Q(s,a) = r + \gamma max Q(s^{'}, a^{'})$$

Deep Q-Network (DQN), introduced by DeepMind, combines Q-Learning with deep neural networks to approximate the Q-function for large state spaces.

Procedure:

1. Install Required Libraries
```
pip install gym torch numpy
```
2. Import Libraries
```
import torch
import torch.nn as nn
import torch.optim as optim
import numpy as np
import gym
import random
```
3. Create Environment
```
env = gym.make("CartPole-v1")
state_dim = env.observation_space.shape[0]
action_dim = env.action_space.n
```
4. Define Deep Q-Network
```
class DQN(nn.Module):
    def __init__(self, state_dim, action_dim):
        super(DQN, self).__init__()
        self.fc = nn.Sequential(
            nn.Linear(state_dim, 128),
            nn.ReLU(),
            nn.Linear(128, action_dim)
        )

    def forward(self, x):
        return self.fc(x)
```
5. Initialize Model and Optimizer
```
model = DQN(state_dim, action_dim)
optimizer = optim.Adam(model.parameters(), lr=0.001)
criterion = nn.MSELoss()
```
6. Training Loop
```
gamma = 0.99
epsilon = 1.0
episodes = 100

for episode in range(episodes):
    state = env.reset()[0]
    state = torch.FloatTensor(state)

    total_reward = 0
```

```
done = False
while not done:
    if random.random() < epsilon:
        action = env.action_space.sample()
    else:
        with torch.no_grad():
            q_values = model(state)
            action = torch.argmax(q_values).item()

    next_state, reward, done, _, _ = env.step(action)
    next_state = torch.FloatTensor(next_state)

    target = reward + gamma * torch.max(model(next_state)).item() * (1 - int(done))
    prediction = model(state)[action]

    loss = criterion(prediction, torch.tensor(target))

    optimizer.zero_grad()
    loss.backward()
    optimizer.step()

    state = next_state
    total_reward += reward

epsilon = max(0.01, epsilon * 0.995)
print(f"Episode {episode+1}, Reward: {total_reward}")
```

Observation:

- Initially, the agent performs poorly due to random exploration.
- Over episodes, cumulative reward increases.
- The agent learns to balance the pole successfully in the CartPole environment.
- Epsilon decay improves exploitation over time.

Conclusion:

The experiment successfully implemented a Deep Q-Network to train an intelligent agent in a simulated environment. The agent learned to maximize rewards through interaction and neural network-based Q-value approximation.

This experiment demonstrates the integration of deep learning and reinforcement learning to create decision-making agents, forming the foundation of modern AI systems used in robotics, gaming, and autonomous systems.